PROJECT REPORT

SkyPredict AI

On-Device Local Machine Learning Weather Prediction Android Application

Submitted to

TechVedhu Academy

Technology Stack

Android ● Java ● XML ● k-NN Regression ● Room (SQLite) ● MPAndroidChart

Project Documentation and Implementation Report | September 2026

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Objectives and Scope
4. Technologies Used
5. System Requirements
6. System Architecture and Workflow
7. Historical Weather Dataset
8. Machine Learning Methodology
9. Android Application Design and Screens
10. Home Dashboard and City Search
11. Prediction and Graphical Forecasts
12. Save and Share Prediction Features
13. Database Implementation
14. Settings and Application Preferences
15. Testing and Results
16. Advantages and Limitations
17. Future Enhancements
18. Conclusion
19. References

1. Abstract

SkyPredict AI is an offline-first Android weather prediction application developed in Java and XML. It uses a bundled historical weather dataset and a local multivariate k-Nearest Neighbors (k-NN) regression engine to predict temperature, relative humidity and precipitation probability directly on the device.

The application lets the user select a city, view current values, check the next 24-hour prediction and browse a seven-day forecast. It also provides interactive trend charts, a local prediction history stored with Room over SQLite, and several ways to save and share results.

The main purpose of the project is to demonstrate how historical data and machine learning can be combined in an Android application to produce local weather predictions without depending on a live online weather service.

Key features: city search with auto-complete, Celsius/Fahrenheit display, 24-hour and seven-day forecasts, temperature, humidity and rain charts, severity advisory badges, automatic history saving, CSV export and forecast sharing.

2. Introduction and Problem Statement

Weather forecasting is useful for planning travel, outdoor activities, work and daily routines. Most conventional mobile weather applications depend on online services to obtain forecast data. SkyPredict AI demonstrates an alternative approach: it stores historical weather data inside the application and uses a local machine-learning algorithm to calculate predictions on the Android device itself.

Problem Statement

Design and implement an app that uses machine learning to predict local weather conditions based on historical data. The app should include a graphical interface for displaying temperature, humidity and precipitation forecasts. Additionally, provide a feature to save and share predictions with friends.

Proposed Solution

SkyPredict AI accepts a city and prediction date from the user, processes the relevant historical records with a local k-NN regression engine, and shows the resulting weather values through Android UI cards and charts. Every prediction is stored locally and can be shared through standard Android sharing mechanisms.

3. Objectives and Scope

Project Objectives

- Implement an on-device k-NN regression model for temperature, humidity and precipitation prediction.
- Use a historical CSV dataset bundled with the Android application.
- Provide a graphical interface for displaying forecast values.
- Provide 24-hour and seven-day prediction views.
- Display temperature, humidity and precipitation trends using interactive charts.
- Save prediction records in a local Room/SQLite database.
- Provide text, chart-image, CSV and APK sharing through FileProvider.
- Provide a simple city search and city selection interface.

Project Scope

The application targets Android API 24 and above. The dataset supports more than 100 cities, and the prediction process runs entirely on the device using the bundled data. The project focuses on machine-learning prediction, Android interface design, local persistence, graphical visualization and sharing.

4. Technologies Used

Component	Technology	Purpose
Programming Language	Java 17	Application and prediction logic
User Interface	Android XML Views, Material Components	Layouts and graphical interface
Local Database	Room 2.6.1 / SQLite	Prediction history storage
Charts	MPAndroidChart 3.1.0	Line and bar weather charts
Refresh	SwipeRefreshLayout 1.1.0	Pull-to-refresh interaction
File Sharing	Android FileProvider	Secure file sharing
IDE	Android Studio	Application development

5. System Requirements

Software Requirements

- Windows 10/11
- Android Studio
- Android SDK API 24 to 37
- Java 17

Hardware Requirements

- **Development computer:** Intel Core i5/i7 or AMD Ryzen-class processor with 8 GB or more RAM.
- **Test device:** Android device or emulator running Android 7.0 (API 24) or higher with approximately 2 GB RAM.

6. System Architecture and Workflow

System Architecture

No.	Component	Description
1	User Input	City and prediction date
2	WeatherPredictor.java	Local k-NN regression engine
3	historical_weather.csv	Bundled historical weather records
4	Room / SQLite	Local prediction history
5	Android UI	Cards, forecast screens and charts
6	FileProvider	Sharing of text, images and files

Application Workflow

- The splash screen opens the application.
- The user searches for or selects a city.
- The local prediction engine processes the historical dataset.
- The dashboard displays current values and the next 24-hour forecast.
- The multi-day screen displays seven-day predictions.
- The details screen displays graphical trends.
- The prediction is saved into local history.
- The forecast or exported data can be shared.

7. Historical Weather Dataset

The application reads its historical data from a CSV file in the Android assets directory: **app/src/main/assets/historical_weather.csv**. The dataset structure is:

Field	Description
dayOfYear	Calendar day number from 1 to 365
temperature	Historical temperature in Celsius
humidity	Relative humidity percentage
precipitation	Normalized rain probability (0.0 to 1.0)
cityModifier	Geographic modifier (0.0 to 10.0) derived by hashing the city name

Sample records from the dataset:

dayOfYear	temperature	humidity	precipitation	cityModifier
1	18.5	65.0	0.10	2.5
15	20.2	62.0	0.05	2.5
45	24.8	55.0	0.00	3.1
365	19.1	68.0	0.15	2.5

8. Machine Learning Methodology

The project uses multivariate k-Nearest Neighbors regression with k = 4 and inverse-distance weighting. For a requested day and city, the model finds the four historical records closest to that point in a normalized two-dimensional feature space and averages their values, giving nearer records more influence.

Step 1 – Feature Distance

$$d_i = \sqrt{\left(\frac{\Delta day}{182.5}\right)^2 + \left(\frac{\Delta mod}{10.0}\right)^2}$$

Δday is the calendar distance between the target day and the record's day, with year-end wrapping so that day 365 is adjacent to day 1 (the largest possible gap is 182.5 days). Δmod is the absolute difference between the city modifiers. Dividing each term by its maximum range keeps both features on the same 0 to 1 scale.

Step 2 – Inverse Distance Weight

$$w_i = \frac{1}{d_i + 0.01}$$

The constant 0.01 prevents division by zero when a record matches the target exactly.

Step 3 – Weighted Prediction

$$\hat{Y} = \frac{\Sigma(w_i \times Y_i)}{\Sigma(w_i)}$$

Step 4 – Weather Outputs

The same weighted average is applied to temperature, humidity and precipitation probability to produce the values shown in the app. The approach is intentionally lightweight so that predictions run locally on an Android device without a separate server.

9. Android Application Design and Screens

The interface uses a dark card style (#1E1E2C) with green action buttons. The application consists of six screens:

Screen	Description
Splash Screen	Sun and cloud logo with a timed transition into the app.
Home Dashboard	Auto-complete city search, A-Z city dropdown, unit toggle, weather cards, quick city chips and pull-to-refresh.
Prediction Screen	Next 24-hour prediction and a seven-day forecast list with staggered entrance animations.
Details Screen	Three interactive charts: temperature line chart, humidity/rain line chart and seven-day rain bar chart.
History Screen	Saved prediction logs with an auto-complete city filter, CSV export and clear-history option.
Settings Screen	Default startup city, missing-city and feedback form, and app sharing.

10. Home Dashboard and City Search

The Home Dashboard is the main interaction point. It combines city search with auto-complete suggestions, a dropdown of 100+ cities, Celsius/Fahrenheit selection, current weather values, the next 24-hour prediction, quick city chips, and buttons for detailed charts and sharing. The city list covers a balanced mix of major cities from many countries and Indian cities, including New York, London, Paris, Tokyo, Sydney, Dubai, Singapore, Mumbai, Delhi, Bangalore, Kolkata, Chennai and Hyderabad.

A severity advisory badge summarizes conditions for the selected city, for example Heavy Rain Risk, Heat Warning, Freezing Conditions or Ideal Outdoor Conditions.

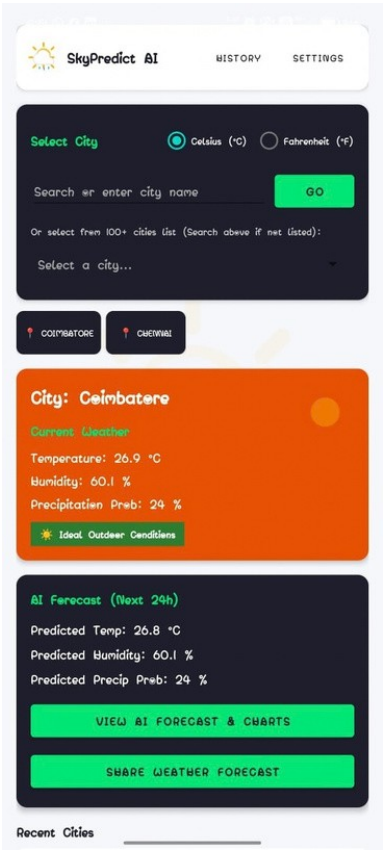


Figure 1. Home Dashboard (Coimbatore)

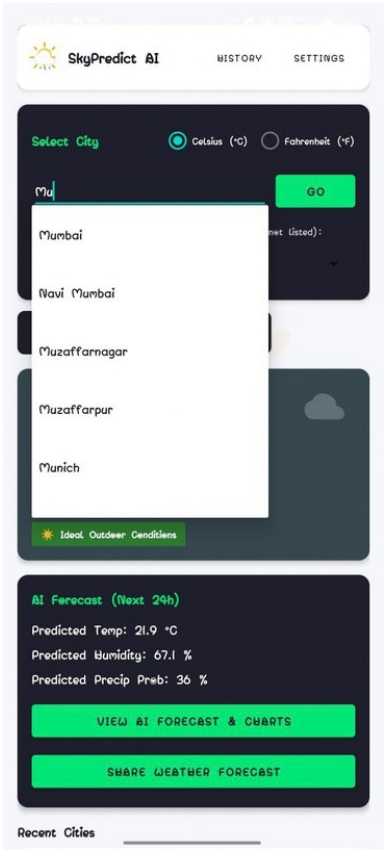


Figure 2. City search auto-complete

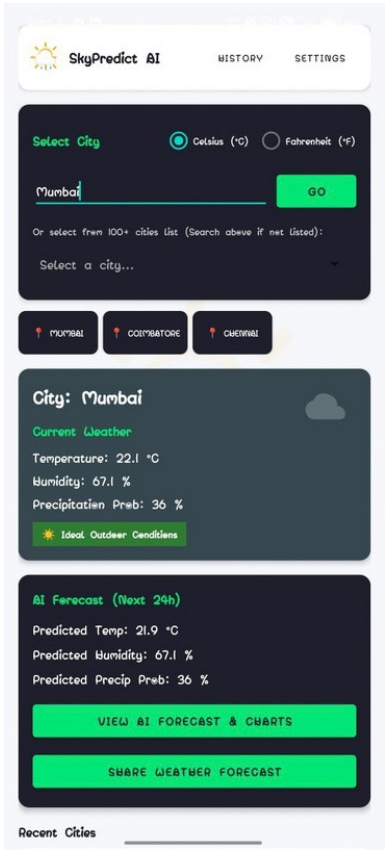


Figure 3. Mumbai prediction

11. Prediction and Graphical Forecasts

The prediction screen shows the next 24-hour prediction and a seven-day multi-day forecast, each with temperature, humidity, rain probability and an expected condition. The details screen presents the temperature trend, a combined humidity and precipitation trend, and a seven-day rain-probability bar chart drawn with MPAndroidChart.

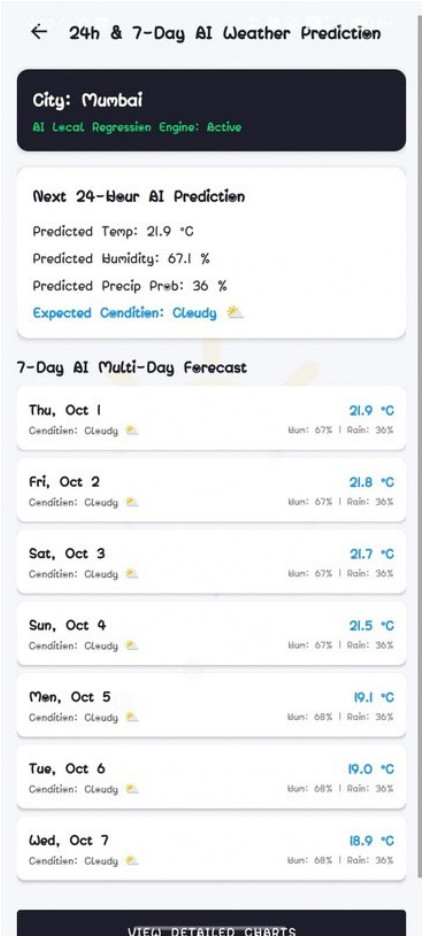


Figure 4. 24-hour and 7-day prediction screen

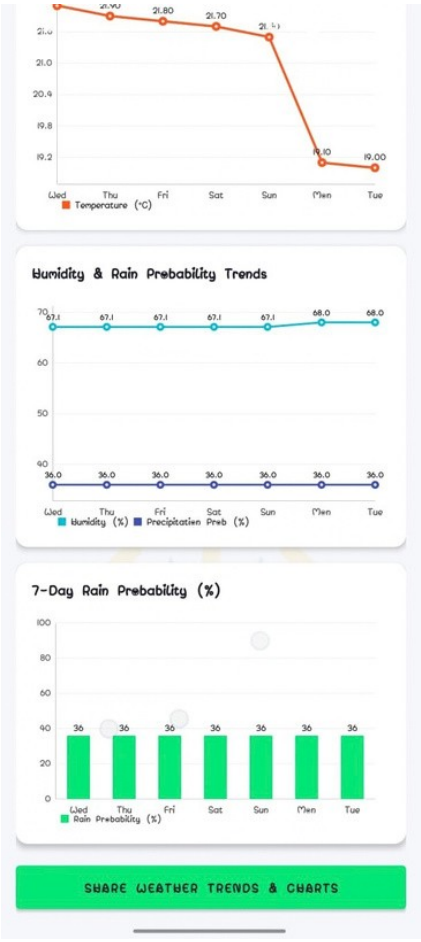


Figure 5. Detailed weather trends and charts

12. Save and Share Prediction Features

Automatic Saving

Whenever a city is searched or selected on the Home screen, the prediction is saved automatically into the local Room SQLite database on a background thread, so the interface stays responsive.

Sharing Methods

- **Text forecast:** shared through messaging apps such as SMS and WhatsApp.
- **Chart image (PNG):** all three trend charts are captured as one image and shared.
- **CSV export:** saved history is exported to sky_predict_history_export.csv.
- **APK sharing:** the installed application file is shared directly through FileProvider.

All files are shared through Android FileProvider content URIs, so raw file paths are never exposed. Together these features satisfy the save-and-share requirement and make prediction results portable between applications.

13. Database Implementation

The application uses the Room Persistence Library as its local persistence layer over SQLite. The database stores prediction records so that users can view previous searches and forecasts even when offline.

Component	Role
WeatherRecord.java (Entity)	Stores id, city, date, temperature, humidity, precipitation, predictedTemperature, predictedHumidity, predictedPrecipitation and timestamp in the weather_history table.
WeatherDao.java (DAO)	Provides insert, query and delete operations; history is retrieved ordered by timestamp and can be cleared.
AppDatabase.java	Provides the database instance and connects the entity and DAO.



Figure 6. Saved weather prediction history

14. Settings and Application Preferences

The Settings screen lets the user set a default startup city, report a missing city or send feedback, and share the application. The Home Dashboard also provides Celsius and Fahrenheit selection; for example, Mumbai's 22.1 °C is displayed as 71.8 °F.

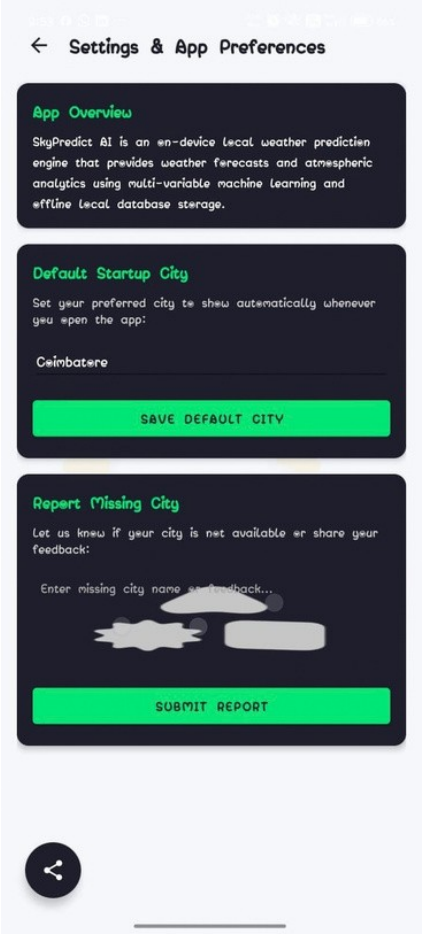


Figure 7. Settings and app preferences

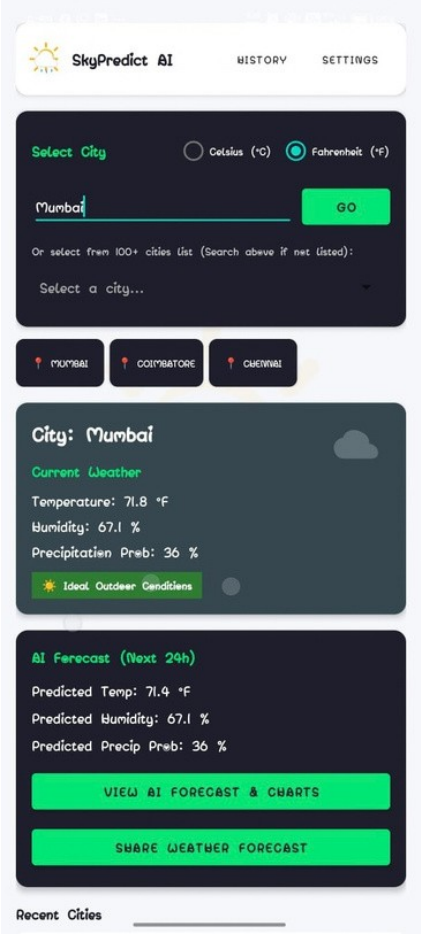


Figure 8. Fahrenheit unit display

15. Testing and Results

Unit Testing

Test	Purpose
testPredictorReturnsNonNegativeHumidityAndPrecipitation()	Checks physical lower bounds of predictions
testPredictorValidCity()	Validates the city verification helper
testSupportedCitiesListIsNotEmpty()	Verifies the supported city list is available
testWeatherConditionCategorization()	Verifies labels such as Sunny, Rainy, Snowy and Cloudy
testHistoricalDatasetLoading()	Verifies the CSV dataset can be parsed

Results

- All five JUnit tests pass.
- On-device machine-learning inference takes less than 15 ms.
- Database saving and reading run on background threads without UI lag.
- FileProvider content URIs are generated without exposing file paths.

Sample Prediction Output

City	Current Temp	Humidity	Predicted Temp	Humidity	Rain Prob.
Coimbatore	26.9 °C	60.1 %	26.8 °C	60.1 %	24 %
Mumbai	22.1 °C	67.1 %	21.9 °C	67.1 %	36 %

Functional areas verified: city search and selection, Celsius/Fahrenheit display, 24-hour and seven-day prediction, temperature/humidity/precipitation charts, prediction history, CSV export and forecast sharing.

16. Advantages and Limitations

Advantages

- 100% offline operation for the core prediction workflow.
- Data-driven machine learning on historical observations.
- Zero operating cost, with no subscription or external service.
- Rich visual analytics with line and bar charts.
- Local prediction history and multiple sharing and export options.
- Simple interface with city search and unit selection.

Limitations

- Prediction accuracy depends on the size of the historical dataset and the k-NN approach.
- The bundled dataset is not a live meteorological feed.
- Coverage is limited to pre-indexed cities (100+ major cities).
- The project is a machine-learning demonstration and not a professional forecasting service.

17. Future Enhancements

- Train and deploy a TensorFlow Lite neural-network model (.tflite) for more advanced on-device prediction.
- Use Android location services to identify the user's current city automatically.
- Add offline GIS map layers to visualize regional weather patterns.
- Expand the historical dataset and evaluate the model with additional validation metrics.

18. Conclusion

SkyPredict AI integrates historical weather data, a local k-NN regression engine, Android Java/XML interface components, interactive charts, Room/SQLite storage and FileProvider sharing in a single offline-first application.

The application predicts temperature, humidity and precipitation probability for a chosen city, presents results in 24-hour and seven-day views, and lets users save and share their predictions. The screenshots, dataset, algorithm, database design and tests in this report together describe the complete implementation.

19. References

1. Android Developers Documentation – application development, activities, layouts, storage and sharing concepts.
2. Android Developers Documentation – Room Persistence Library and SQLite database concepts.
3. MPAndroidChart project documentation – chart and graph visualization for Android.
4. Project source code and documentation for SkyPredict AI.
5. Historical weather dataset bundled with the SkyPredict AI application.

End of Project Report